Decoupled Information-Set MCTS with a Card-Value Model for Sushi Go!

Halit Ozger
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 2]
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 3]
Student no. [to be completed]
Queen Mary University of London
London, UK

***Abstract*—In Sushi Go!, players pick cards simultaneously from hidden hands that are passed around, which basic Monte Carlo tree search (MCTS) does not model. Starting from BasicMCTS of the Tabletop Games (TAG) framework, we added a scoring model that drives evaluation and rollouts, round-end rollouts, max-n backups, decoupled selection for the simultaneous choices and information-set search. On held-out deals the agent wins 52.6% of 500 three-player games at 100 ms against TAG's information-set MCTS and our previous agent (chance 33.3%), [pending]% at 900 ms and 44.4% with four players (chance 25%). Ablations show that the scoring model and the decoupled information-set tree matter most; the tree's advantage grew with the budget, though not significantly, and four-player settings cost 7.7 percentage points with three players.**

Keywords*—*Monte Carlo tree search, information sets, simultaneous moves, card games, Sushi Go!, TAG

I. Introduction

Tabletop games combine chance, hidden information and several interacting players within small rule sets; the Tabletop Games framework (TAG) [1] gives many of them a common interface so that agents can be compared. Monte Carlo tree search (MCTS) [2] is the usual starting point for such agents, but its basic form assumes that the state is fully observed and that players move one after another [3]. *Sushi Go!*, one of TAG's games, breaks both assumptions: every player secretly picks one card from its hand, all players reveal their cards together and pass the rest of their hands on, and after three rounds the highest score wins. Klang et al. [4] introduced the game to TAG and assessed simultaneous action selection and complete information in it; the current version, which we use, shows an agent only the cards it has seen.

Four properties make the game hard for a search agent. (i) The opponents' hands are hidden until they are passed on. (ii) Moves are simultaneous: no card can be chosen in reaction to an opponent's card of the same turn. (iii) Many points are delayed and conditional – sets pay out only when completed, Maki rolls at the end of the round and Pudding at the end of the game – so short simulations miss them and long ones are noisy. (iv) Hands of nine (three players) or eight cards (four players) give 729 or 4 096 joint moves per turn (more with Chopsticks), with 0.1–1 s per decision. TAG's BasicMCTS addresses none of the first three: it searches the one shuffled sample of the hidden cards it receives as if it were the true state, treats the players of a turn as moving one after another and evaluates random rollouts by the player's own score.

We ask which adaptations of MCTS to these properties make an agent stronger (RQ1), and how their value changes with the time per decision (RQ2) and the number of players (RQ3). We extend BasicMCTS in steps, keeping every modification as a switch, and test each in experiments of 500 games per condition (250 in some long-budget development runs). Our contributions are:

- a model of Sushi Go!'s scoring that estimates every player's final score and every card's marginal value, used both to evaluate states and as the rollout policy of all players;
- its combination on BasicMCTS with round-end rollouts, max-n backups, decoupled UCT and information-set search, compared with TAG's own decoupled information-set MCTS;
- evidence that settings tuned for four players hurt with three, and a diagnosis of why extra time helps little with four (a wide, shallow joint-move tree);
- leave-one-out and factorial ablations, held-out validation and negative results (162 runs, 69 350 games).

II. Background

A. TAG and Sushi Go!

TAG [1] splits a game into a state, a forward model and parameters; an agent receives a copy of the state, in which every card it has not seen is reshuffled, and uses the forward model to simulate moves. In TAG's Sushi Go! the 108-card deck holds Tempura (5 points per pair), Sashimi (10 per triple), Dumplings (1, 3, 6, 10 or 15 points for one to five), Salmon, Squid and Egg Nigiri (2, 3 and 1 points, tripled on an earlier Wasabi), Maki rolls with one to three icons (6 and 3 points for the most and second-most icons of a round), Chopsticks (two cards in a later turn) and Pudding (+6 for the most and, with three or more players, −6 for the fewest at the end of the game). Each round deals $12 - n$ cards to each of n players. As the hands rotate, every hand of a round has been seen after $n - 1$ passes, so hidden information is concentrated at the start of each round, while the value of a card depends on what the others will take from the hands we pass on.

B. Monte Carlo Tree Search

MCTS [2] builds a search tree by repeating selection, expansion, simulation (rollout) and backpropagation. Upper Confidence bounds applied to Trees (UCT) [5] selects the child that maximises the upper confidence bound (UCB1) $Q + K\sqrt{(\ln N / n)}$, where Q is the child's mean reward, n its visits and N those of its parent. Domain knowledge can enter through heuristic rollout policies or priors such as progressive bias [2]. A stronger rollout policy does not guarantee better estimates, however: Silver and Tesauro [6] showed that rollouts should be balanced, i.e., give unbiased value estimates, rather than be played by a strong policy.

C. Hidden Information, Simultaneous Moves and Several Players

Determinisation, or perfect-information Monte Carlo (PIMC) sampling, samples the hidden information, searches each sample as if it were the true state and combines the results. Long et al. [7] related its success in card games to properties such as how quickly hidden information is

All sections, tables and figures of this paper were drafted with Claude [15]; the prompts are given in the statement on the use of generative AI at the end of the paper.

revealed during play, which in Sushi Go! happens within $n - 1$ turns. Its known flaws are strategy fusion – the search assumes that it can act differently in states it cannot tell apart – and non-locality [8]. Information-set MCTS (ISMCTS) [3] avoids strategy fusion by searching one tree over the searcher's information sets and sampling the hidden information anew in every iteration. In its single-observer form the opponents' choices in the tree are not conditioned on their own private information; the multiple-observer form models it with one tree per player at a higher cost [3], an idea TAG's advanced MCTS agent offers as MultiTree MCTS [9].

Simultaneous moves are usually handled by decoupled UCT (DUCT): every player chooses its own action at a node from its own bandit statistics and the joint action leads to a child [10]. UCB-based DUCT need not converge to an equilibrium, so Lanctot et al. [10] compared it with selection by the EXP3 bandit algorithm and by regret matching, and Tak et al. [11] found that the best variant depends on the game; both studies used two-player games such as Goofspiel. In multi-player games, UCT with max-n backups, in which every player maximises its own value, computes a mixed-strategy equilibrium and played at least as well as earlier algorithms in Sturtevant's experiments [12].

MCTS has also been applied to the drafting game 7 Wonders, which passes hands in the same way [13]. TAG's advanced MCTS agent already offers information sets, one tree per player and a decoupled option; we build on BasicMCTS, which has none of them, so that the contribution of each idea can be measured. How these ideas combine with domain knowledge in a multi-player drafting game, and how their value depends on the time budget and the number of players, has not been measured; the earlier Sushi Go! study in TAG [4] focused on simultaneous action selection and complete information.

III. Method

Our agent, SushiMCTS, is a modified copy of TAG's BasicMCTS (package *players.basicMCTS*) and uses no code of TAG's advanced MCTS agent. Every modification is a switch in its parameter class (Table I). We compare the final agent (v4) with two earlier versions, v2 (H, A, B, D) and v3 (H, B, D, P, R, M), whose tree treats the players of a turn as moving in sequence (Fig. 1). The search techniques come from the literature; our own element is the card-value model shared by the evaluation and every player's simulation.

TABLE I. Modifications of BasicMCTS (✓ = used in the final agent with 3 / 4 players)

Id	Modification	3 p / 4 p
H	heuristic (2)–(3) from model (1)	✓ / ✓
B	ε-greedy card-value rollouts for every player	✓ / ✓
R	rollouts end when the round is scored	✓ / ✓
P	one branch per distinct card	✓ / ✓
M	max-n backups [12]	✓ / ✓
S	decoupled UCT, one node per turn [10]	✓ / ✓
I	single-observer information-set search [3]	✓ / ✓
X0	four-player card constants [a]	– / ✓
W2	win-gap value for 4 players (Section III-D)	– / ✓ [b]
F, L	fast simulator, 8 rollouts per leaf	– / ✓ [b]
A	progressive bias [2]	dropped in v3
D	ensemble of up to 4 determinised trees [7]	replaced by I

[a] (σ, *w*, α, *T*, β, T_m, κ, δ, π, ν, γ) = (15, 0.7, 0.5, 6, 0.5, 3, 0.25, 1.5, 1.25, 3, 3), default (15, 0.7, 0.5, 4, 1, 0, 0, 0, 1, 2, 0): β scales a Sashimi triple still needing two cards; the Maki standing gets weight $1/(1 + r/T_m)$, κ points per icon the rest; δ adds the next Dumpling's points; π scales the Pudding standing; a set is likely only if γ cards per missing card remain in the hands. [b] at 500 ms or more.

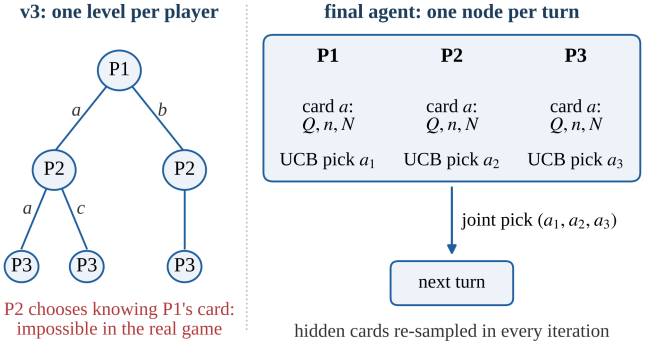


Fig. 1. The turn node of the final agent (right) and the sequential tree of v3 (left). The final agent re-samples the cards it cannot see in every iteration (I), lets every player pick by UCB on its own card statistics (S, P), plays the joint pick, rolls out with card-value policies until the round is scored (B, R) and backs every player's value (H) up to the card it chose (M); after the time budget it plays its most visited root card.

A. Card-Value Model and Heuristic (H)

For a state s and player p the model estimates the final score

$$\hat{S}_p = S_p + \alpha U_p + M_p + w_t P_p, \quad (1)$$

where S_p is the banked score (TAG scores Tempura, Sashimi, Dumplings and Nigiri when they are revealed), U_p the value of unfinished sets, of each unused Wasabi (2ν for the Nigiri it can still triple, ν = 2) and of Chopsticks (1), each weighted by a completion factor $\min(1, (r - m + 1)/T)$ (0 if $r < m$) when m more cards are needed with r picks left, M_p the Maki points p would get if the round ended now, and P_p the Pudding points if the game ended now, weighted by $w_t = t/3$ in round t (α = 0.5, T = 4). The value of s for player i is

$$H_i(s) = \tanh(\Delta_i / \sigma), \quad (2)$$

$$\Delta_i = w(\hat{S}_i - \max_{j \neq i} \hat{S}_j) + (1 - w)(\hat{S}_i - \mathrm{mean}_{j \neq i} \hat{S}_j), \quad (3)$$

with σ = 15 and w = 0.7: the first term rewards beating the leader, since only first place wins, and the second keeps a gradient when the agent is far ahead or behind. The marginal value $mv_p(c)$ of a card c is the change of $\hat{S}_p$ when p plays it.

B. Simulation (B, R)

B: every player plays its card with the highest *mv*, or a random card with probability ε = 0.2 (with Chopsticks, the second card is chosen in the same way); the same policy models the opponents. **R:** a rollout stops when the current round has been scored; only Pudding carries over, which (1) captures, so the leaves are valued by banked points and the Pudding standing.

C. Tree (P, M, S, I)

P: identical cards lead to identical states, so a node has one branch per distinct card. **M:** a node stores one value per player and every player maximises its own value [12]. **S:** one node represents a whole turn; at a node every player p picks a card by

$$a_p = \mathrm{argmax}_a \, [Q_p(a) + K\sqrt{(\ln N_p(a) / n_p(a))}] \quad (4)$$

from its own statistics, trying untried cards first; the joint choice is played in the forward model and leads to the child of that joint action [10]; in v3's tree a later player "saw" the card an earlier player had just chosen (Fig. 1). **I:** every iteration starts from a new sample of the cards we cannot see and walks one shared open-loop tree, whose nodes store statistics but no states [3]. Statistics are keyed by card type; $n_p(a)$ counts how often p chose a at the node and the availability count $N_p(a)$ how many iterations reached the node with a in p's sampled hand. I replaces the

determinisation ensemble D of v2 and v3, which split the budget between up to four independent trees. With three players $K = 0.5$.

D. Four Players

With four players the agent switches parameters. It uses the card constants X0 (Table I), found by a search-free coordinate tuner that plays greedy card-value policies against each other. From 500 ms it also uses **W2** – $w = 1$; finished games valued by $\tanh((S_i - \max_{j \neq i} S_j)/2)$ with Pudding as tie-break; σ multiplied by $(R/2)^{0.5}$ for R rounds left; opponents in the backup valuing their lead over the others' mean – and **F**, TAG's rules re-implemented on integer arrays and checked move by move against the forward model, whose speed is spent on **L** = 8 rollouts per leaf, with $K = 1$. The competition files use this agent with 900 ms (1 s limit) and 430 ms (500 ms limit) of CPU time and wall-clock guards.

E. Experimental Setup

TABLE II. EXPERIMENTAL SETUP

Item	Setting
Framework	TAG as distributed with the module, Sushi Go! with its default rules; OpenJDK 21
Opponents	v2: our previous agent (H, A, B, D, K 1.41). TAG ISMCTS: TAG's SushiGo_3+P_HiBudget file (set for 400 ms) at our budget: one tree per player [9], UCB-Tuned, K 1, random rollouts to the end, score heuristic. TAG DUCT: TAG's SushiGoDUCT file: decoupled UCT, K 1, information sets, lead heuristic, 30-move rollouts. BasicMCTS (K 1.41, 10-move rollouts), Rolling Horizon Evolutionary Algorithm (RHEA; horizon 10, population 10), One-Step Look-Ahead (OSLA, score heuristic)
Budget	CPU time per decision, equal for all agents: 100 ms (most development), 500 and 900 ms
Seating	our agent in seat g mod n in game g; the other seats alternate between the two opponents (4 players: 2 + 1, swapped every 4 games)
Deals	game g uses seed $s + g$; since v4 a fixed deck order is shuffled with it (equal s, equal deals: paired runs); earlier runs used TAG's own deal (independent samples)
Seeds	development s = 1, 2 (v3 also 1001–3001); ablation and last study s = 7; held-out validation s = 100 000, disjoint from all other deals
Games	500 per condition; 250 in development runs at 900 ms and with four players at 500 ms (48–73 s of CPU time per game)
Hardware	18-core laptop (6–16 games in parallel); last study and validation on a 4-core machine (4 in parallel)
Records	agent files, per-game results and scripts of every run: available from the authors

Table II summarises the setup. Each game is an independent trial whose outcome for our agent is 1 for a win, $1/k$ for a k-way shared first place and 0 otherwise. Win rates are reported with Wilson 95% confidence intervals (CI) [14]; the score margin (our score minus the best opponent's) as mean ± 1.96 standard errors (SE); differences between runs as the mean of the per-game differences on the same deals ± 1.96 SE (for independent samples, the difference of the means with the two-sample SE) and z = difference/SE, in percentage points (pp) for win rates. With 500 games a win rate near 50% has a 95% interval of ±4.4 pp, and a difference between two runs about ±5.7 pp even when paired, because the outcomes of two agents on the same deal correlate only weakly (0.0–0.2); the score margin, about twice as sensitive, served to screen ideas. We report unadjusted intervals and state where a conclusion survives a Holm correction. We expected (H1) every component to improve both measures, (H2) the decoupled information-set tree to matter more at longer budgets and (H3) the best settings to depend on the number of players.

IV. RESULTS

A. Final Agent (RQ1)

TABLE III. FINAL AGENT ON HELD-OUT DEALS AND AGAINST OTHER AGENTS

Setting	Opponents	n	Win % [95% CI]	Margin
3p/100	v2, ISMCTS	500	52.6 [48.2, 56.9]	+0.73 ± 0.46
3p/500	v2, ISMCTS	500	pending	pending
3p/900	v2, ISMCTS	500	pending	pending
4p/100	v2, ISMCTS	500	44.4 [40.1, 48.8]	−0.61 ± 0.43
4p/500	v2, ISMCTS	500	pending	pending
3p/100	DUCT, ISMCTS	500	55.0 [50.6, 59.3]	+0.72 ± 0.40
3p/100	ISMCTS × 2 [a]	300	54.2 [48.5, 59.7]	+1.66 ± 0.69
3p/100	BasicMCTS, RHEA [a]	500	84.2 [80.7, 87.1]	+6.68 ± 0.60
3p/100	OSLA, BasicMCTS [a]	500	87.2 [84.0, 89.8]	+9.74 ± 0.75

Setting: players / ms per decision. Held-out seed block 100 000, except [a] seed block 7 (used for no decision about these opponents). Margin: our score minus the best opponent's, ± 95% CI. Chance: 33.3% (3 players), 25% (4 players).

Table III reports the final agent. On the held-out deals it wins 52.6%, [pending] and [pending] of three-player games at 100, 500 and 900 ms against v2 and TAG's ISMCTS, and 44.4% and [pending] of four-player games at 100 and 500 ms. On seed block 7, whose deals overlap the development deals, it had won 55.0% at 100 ms (58.1% when re-run on the same machine), 2.4 ± 6.1 pp and 5.5 ± 6.1 pp more; neither difference is significant. TAG's DUCT agent, with decoupled selection and information sets but neither our card-value model nor round-end rollouts, wins only 21.0% of its seats when it replaces v2. BasicMCTS, RHEA and OSLA win at most 12.8% of their seats; OSLA wins none, as its copy of the state hides the other players' choices, so its look-ahead never reveals a card.

B. Ablation (RQ1)

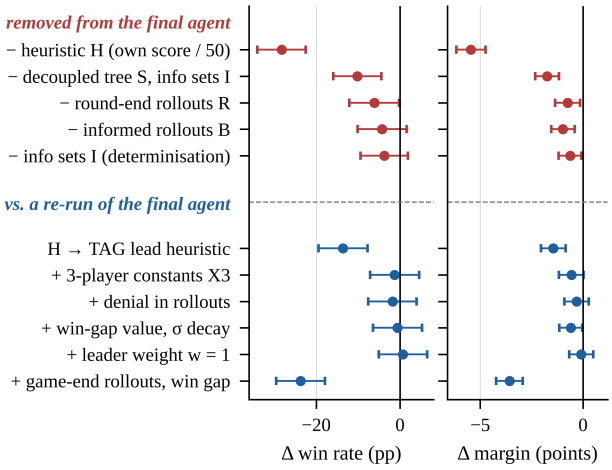


Fig. 2. Paired effects with three players at 100 ms (seed 7, 500 games each), ± 95% CI. Top: one component removed from the final agent (base: final agent, 55.0%). Bottom: the lead-heuristic ablation and the changes of our last study, against a re-run of the final agent on the 4-core machine (58.1%).

Fig. 3 (top) removes one component at a time from the final agent. Every removal lowers both measures; on the score margin all five differences survive a Holm correction, on the win rate only those of H and of S + I (−R: $z = 2.0$, −B: 1.4, −I: 1.3), so H1 holds for the margin and only partly for wins. Without H the leaves are valued by TAG's default heuristic, the own score divided by 50, which ignores the opponents' scores and the Pudding standing and does not match the scale of K: this costs 28.3 ± 5.8 pp. TAG's lead heuristic (own score minus the best opponent's, K rescaled to 7.5) in place of H costs 13.6 ± 5.9 pp ($z = 4.5$): valuing the lead recovers about half of H's effect; the rest (Pudding

standing, mean-opponent term, bounded scale) was not separated. S and I together are worth 10.2 ± 5.7 pp (z = 3.5) but I alone only 3.8 ± 5.7 pp, as Long et al.'s analysis [7] suggests for a game that reveals its hidden information within $n - 1$ turns. P and M cannot be removed (decoupled selection needs one value per player; I's card-type statistics merge identical cards), but a 2^4 factorial on v3's components (16 × 500 games) gave main effects of +8.7 ± 2.1 pp for R (z = 8.1), +5.2 ± 2.2 for K = 0.5 instead of 1.41 and +4.1 ± 2.0 for M, with an M × K interaction of +3.2 ± 2.2: max-n backups need less exploration. Adding progressive bias back to v3 did not help (−1.2 ± 4.3 pp).

The changes of our last study did not help (Fig. 3, bottom). Card constants tuned for three players (X3) make a far stronger greedy policy – they win 45.0% of greedy games against two X0 players, whereas the final agent's own constants win 7.3% in a pool of their own and X0 players – yet did not strengthen the search (−1.3 ± 5.8 pp), consistent with simulation balancing [6]. Denial (simulated players also value taking the next player's best card), a win-gap value and a leader-only value did not change the win rate either (−1.8, −0.6, +0.7 pp), and rollouts to the game end with a win-gap value cost 23.7 ± 5.8 pp (z = 8.0).

C. Time Budget (RQ2)

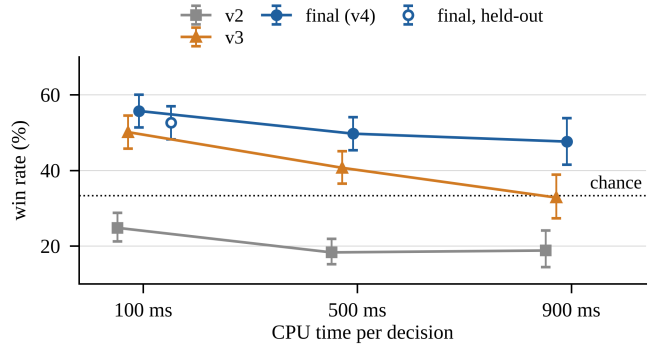


Fig. 3. Win rate with three players against v2 and TAG's ISMCTS by CPU time per decision. Lines: development runs (seed 1, independent samples). Open markers: the final agent and v3 on the held-out deals (seed block 100 000). Bars: Wilson 95% intervals.

In development runs (Fig. 2; independent samples) v3 doubled v2's win rate at 100 ms (24.8% → 50.1%) but lost 17.3 ± 7.2 pp from 100 to 900 ms, the final agent 8.1 ± 7.5 pp; the gain from S and I rose from +5.6 ± 6.1 pp to +14.8 ± 8.4 pp (rise +9.2 ± 10.4 pp (z = 1.7)). We attribute the gain to v3's sequential tree, in which later players react to cards chosen earlier in the same turn: a wrong model that more iterations exploit more. It is not a lack of search: at 900 ms v3 ran 19 696 iterations per decision, the final agent 21 246. Much of the fall of all curves comes from the opponent: TAG's ISMCTS won 23.1%, 33.1% and 37.4% of the development games against the final agent at 100, 500 and 900 ms (33.0%, 40.0% and 46.0% against v3).

D. Four Players and the Limits of Search (RQ3)

With four players the 3-player configuration won 46.4% and 43.5% in two runs at 100 ms and 38.4% at 500 ms (development; chance 25%). The four-player changes gave no significant gain at their own budgets: X0 added +4.3 ± 4.3 pp at 100 ms (two runs of each pooled); at 500 ms X0 with W2 and K = 1 added +5.8 ± 7.4 pp (two runs pooled) and F + L a further +1.0 ± 7.9 pp, whereas F + L cost 6.6 ± 6.0 pp (z = 2.2) at 100 ms. With three players at 500 ms, X0, W2 and K = 1 cost 7.7 ± 5.4 pp (z = 2.8) (F + L in both arms), so their effect differs between the table sizes by 13.5 ± 9.1 pp (z = 2.9). This supports H3 only one-sidedly (a clear loss with three players, no clear gain with four), and the contrast was chosen after about 40 four-player runs, so it is exploratory. At 500 ms the four-player tree cut to depth 1 lost 17.4 ± 8.2 pp (z = 4.2) and our agent limited to 100 ms 9.0 ± 7.7 pp (z = 2.3), yet 1 000 ms (with K = 1.37) did not help (−4.8 ± 8.1 pp). On 73 positions from random games the 500 ms tree has 704 joint-move children at the root on average, 53% of them visited once, and a mean depth of 1.5; twice the time or rollouts per leaf changed the chosen card in 2.7% and 5.5% of the positions, no more often than a re-run with the same seed (5.5%; 9.6% with another seed).

V. Conclusions and Future Work

We adapted TAG's BasicMCTS to Sushi Go! with a shared card-value model, round-end rollouts, max-n backups, decoupled UCT and information-set search. For RQ1, the heuristic (28.3 pp when removed; a plain lead recovers half of it) and the decoupled information-set tree (10.2 pp) contribute most, and round-end rollouts led v3's factorial (+8.7 pp). On held-out deals the final agent wins 52.6% of three-player games at 100 ms and [pending] at 900 ms against TAG's ISMCTS and our previous agent. For RQ2, the decoupled tree loses less strength than the sequential one at longer budgets (−8.1 against −17.3 pp from 100 to 900 ms), though not significantly. For RQ3, settings tuned for four players cost 7.7 ± 5.4 pp with three players while their gain with four was not significant, and with four players extra time rarely changes the decision of the wide, shallow joint-move tree.

The study has limits. Our opponents are framework agents and our own previous version; the four-player 500 ms development rests on 250-game runs and its contrast with three players is exploratory; the factorial and the development budget runs used independent deals; and CPU-time budgets hide wall-clock effects such as garbage collection. As hand-tuned changes to the model no longer move the agent, future work should learn the evaluation from game outcomes, e.g. by regressing final ranks on features of round-end states. Regret matching or EXP3 could replace UCB in the decoupled selection [10], as the best variant depends on the game [11], and opponent models fitted to the picks of real opponents could replace the shared greedy policy.

Use of Generative AI

We used Claude [15] (Anthropic), through the Claude Code tool, throughout the project: to write and modify the Java code of our agent on top of BasicMCTS, to write and run the experiment and statistics scripts, and to draft all sections, tables and figures of this paper from the logs of our runs; every number is computed from the result files of our runs. The authors take full responsibility for the code and the paper, including any errors. The paper was generated with the prompt (original Turkish, then English translation): *"simdi reponun icindeki butun herseyi incele ve hocaya direk yuklyecegimiz raporuda olusutr repoda ornekler tipler brief hersey var 100 alcagimiz bir rapor olsun her raporu tamaladiginda baska bir agent spawn edip bagimsiz durusstce kontrol taki yuzde yuz alacagana emin olankadar looptan cikma"* – "Now examine everything in the repository and also create the report we will upload directly to the lecturer; the repository has examples, tips, the brief, everything. It should be a report that gets 100. Every time you finish the report, spawn another agent to check it

independently and impartially, and do not leave the loop until you are sure it will get 100%.” Following that prompt, each draft was marked by a second, independent Claude instance against the module's rubric; its written findings, archived with our records, were the prompts for the revisions and for the additional runs (held-out validation, TAG DUCT comparison, lead-heuristic ablation, v3 on the held-out deals, four-player diagnostics). The experiments of the last study were run with the prompt “The lecturer will hold a tournament of our agents in class; as before, improve the agents' performance quickly while following the lecturer's restrictions and recording what we did in the README; you have 4.5 hours, test everything in parallel.” The prompts of earlier development sessions are not reproduced here; they asked Claude to implement and test the modifications of Table I and to run the experiments of Section IV.

REFERENCES

[1] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, “TAG: A tabletop games framework,” in *Proc. AIIDE Workshop on Experimental AI in Games (EXAG)*, 2020.

[2] C. B. Browne *et al.*, “A survey of Monte Carlo tree search methods,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[3] P. I. Cowling, E. J. Powley, and D. Whitehouse, “Information set Monte Carlo tree search,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.

[4] C.-M. E. Klang, V. Enhörning, A. Alvarez, and J. Font, “Assessing simultaneous action selection and complete information in TAG with Sushi Go!,” in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.

[5] L. Kocsis and C. Szepesvári, “Bandit based Monte-Carlo planning,” in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.

[6] D. Silver and G. Tesauro, “Monte-Carlo simulation balancing,” in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 945–952.

[7] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, “Understanding the success of perfect information Monte Carlo sampling in game tree search,” in *Proc. 24th AAAI Conf. Artif. Intell.*, 2010, pp. 134–140.

[8] I. Frank and D. Basin, “Search in games with incomplete information: A case study using Bridge card play,” *Artif. Intell.*, vol. 100, no. 1–2, pp. 87–123, 1998.

[9] J. Goodman, D. Perez-Liebana, and S. Lucas, “MultiTree MCTS in tabletop games,” in *Proc. IEEE Conf. Games (CoG)*, 2022, pp. 292–299.

[10] M. Lanctot, V. Lisý, and M. H. M. Winands, “Monte Carlo tree search in simultaneous move games with applications to Goofspiel,” in *Computer Games (CGW 2013)*, Commun. Comput. Inf. Sci., vol. 408. Cham, Switzerland: Springer, 2014, pp. 28–43.

[11] M. J. W. Tak, M. Lanctot, and M. H. M. Winands, “Monte Carlo tree search variants for simultaneous move games,” in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2014, pp. 1–8.

[12] N. R. Sturtevant, “An analysis of UCT in multi-player games,” in *Computers and Games (CG 2008)*, Lect. Notes Comput. Sci., vol. 5131. Berlin, Germany: Springer, 2008, pp. 37–49.

[13] D. Robilliard, C. Fonlupt, and F. Teytaud, “Monte-Carlo tree search for the game of ‘7 Wonders’,” in *Computer Games (CGW 2014)*, Commun. Comput. Inf. Sci., vol. 504. Cham, Switzerland: Springer, 2014, pp. 64–77.

[14] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[15] Anthropic, “Claude” (large language model, used through the Claude Code tool), 2026. [Online]. Available: https://claude.ai